

VibeOS Architecture: Next-Generation Vibe Coding System for High-Performance Development

The VibeOS Paradigm and the Evolution of AI-Assisted Engineering

The transition from manual software engineering to AI-assisted development—often colloquially referred to as "vibe coding"—demands a fundamental restructuring of the local development environment. Traditional integrated development environments (IDEs) and terminal emulators were architected for synchronous, human-driven keystrokes. In contrast, modern AI-driven engineering is characterized by highly parallelized, asynchronous agentic loops executing across multiple context windows.¹ The developer's role has consequently shifted from a syntax generator to a principal architect and orchestrator of concurrent autonomous processes.

For software architects operating at the highest echelons of web performance, decisions must be driven by data, execution must be ruthlessly efficient, and cognitive overhead must be minimized. The baseline frameworks currently utilized for AI coding provide foundational automation, utilizing basic shell hooks and specialized subagents to isolate context and prevent the pollution of the main conversational window.¹ However, these early frameworks operate under a critical, flawed assumption: they presume the developer is actively monitoring the terminal interface, waiting synchronously for the large language model to complete its reasoning and execution cycles.

When a developer scales their workflow to manage five to fifteen concurrent agent sessions—a standard practice for maximizing throughput—this synchronous monitoring requirement becomes a catastrophic bottleneck.¹ If an agent encounters a blocked operation requiring human permission, or if it successfully completes a complex refactor, it silently halts. The developer is left to manually poll dozens of terminal tabs to determine which process requires attention, destroying the state of "Deep Work" and severely degrading overall system velocity.

VibeOS is engineered to eradicate this bottleneck. Designed as a comprehensive plugin for the Claude Code ecosystem, VibeOS upgrades the terminal environment into a highly interactive, multitasking-aware, and aggressively token-efficient operating system. The architecture completely rejects the paradigm of passive monitoring. Instead, it treats human attention as the most constrained and valuable resource in the development lifecycle. By enforcing strict interrupt protocols, automating retrospective token-efficiency coaching, and

mandating architectural research prior to implementation, VibeOS aligns perfectly with the data-driven, efficiency-obsessed workflows required by elite web performance experts.

Critical Analysis of the Baseline Framework

An exhaustive analysis of the foundational team guide reveals both the strengths and the severe limitations of current AI-coding configurations.¹ The baseline architecture establishes three critical pillars: deterministic hooks, isolated subagents, and proactive context management.¹

The baseline correctly identifies that relying on an AI model to "remember" to execute standard quality checks or format code is fragile and consumes unnecessary tokens.¹ To resolve this, it implements deterministic hooks—shell commands that execute automatically based on specific lifecycle events, such as PostToolUse for code formatting or PreToolUse for blocking dangerous shell commands.¹ Because these hooks execute standard bash scripts, they consume zero AI context tokens, providing a highly efficient mechanism for enforcing project rules.¹

Furthermore, the baseline framework emphasizes the necessity of subagents to maintain token efficiency. By spawning specialized agents (e.g., a test-writer or a product-researcher) in isolated context windows, the main conversation avoids accumulating thousands of tokens of irrelevant research data or documentation.¹ This is further optimized by integrating Model Context Protocol (MCP) servers, which allow the agent to fetch external data on-demand rather than relying on the developer to paste massive documentation payloads into the prompt.¹

Despite these optimizations, the baseline framework exhibits a severe deficiency in state management and developer attention allocation. The baseline's workflow relies heavily on the developer actively guiding the agent through sequential phases: research, specification, design, implementation, and review.¹ While this linear progression ensures quality, it traps the developer in a reactive loop. The framework lacks any mechanism to signal the developer when an asynchronous phase requires intervention.⁶

When a high-performance architect initiates a complex implementation phase that may require thirty minutes of autonomous agent execution, that architect must be free to shift their focus to a Chrome performance profiler, a Figma design file, or a different repository entirely. The absence of a robust, cross-application notification system in the baseline architecture renders true parallelized development impossible, as the cognitive burden of tracking multiple agent states negates the speed advantages of the AI itself.

Phase 1: The Attention Economy and the Interrupt

Protocol

To resolve the cognitive bottleneck inherent in parallel agent execution, VibeOS introduces the "Attention Economy" architecture. This design philosophy posits that the development environment must remain entirely silent and invisible during normal processing, allowing the developer to maintain uninterrupted focus on secondary tasks. The system is permitted to break this silence only when strict, predefined intervention thresholds are met.

This is achieved through the implementation of the "Interrupt Protocol," an active, terminal-integrated notification matrix designed specifically for modern Rust-based terminals such as Warp.⁷

The Notification Hook Architecture

The Claude Code software development kit exposes a dedicated Notification event within its lifecycle hooks.³ This event is triggered internally by the system when the agent enters a blocked state awaiting user input, or when it completes a background operation.⁴ VibeOS leverages this hook within the hooks/hooks.json configuration file, applying strict regex matchers to ensure the execution payload only fires under specific conditions.³

The Interrupt Protocol dictates that native system notifications are dispatched exclusively under the following three conditions:

- 1. **Permission Stalls (permission_prompt):** The agent attempts to execute a shell command, read a sensitive file, or initiate a network request that falls outside the pre-approved operational boundaries defined in .claude/settings.json.¹ The execution is halted, and the system awaits a Y/N authorization from the developer.
- 2. **Task Completion (idle_prompt):** A complex, multi-turn agentic loop successfully resolves its assigned objective and returns control to the standard prompt interface, awaiting the next instruction from the user.⁹
- 3. **Critical Failures (PostToolUseFailure):** The agent executes a tool that returns a fatal exit code, encounters a compilation error that it cannot autonomously resolve after multiple attempts, or the session reaches critical context exhaustion limits, requiring immediate human intervention to salvage the working state.¹⁰

The following table details the mapping of Claude Code lifecycle events to the VibeOS Interrupt Protocol triggers, outlining the required JSON input schemas and the resulting notification logic:

Hook Event	Matcher Regex	JSON notification_type	Trigger Condition	Interruption Priority
------------	---------------	------------------------	-------------------	-----------------------

Notification	permission_prompt	permission_prompt	Agent halted; waiting for explicit Y/N input on a blocked tool call.	Critical - Blocks execution thread.
Notification	idle_prompt	idle_prompt	Agent loop completed successfully; process is idle.	High - Ready for new instruction.
PostToolUseFailure	* (All failures)	N/A (Parsed via exit code)	Tool executed but returned a non-zero exit code or compilation failure.	High - Requires architectural debugging.
PreToolUse	* (All tools)	N/A	Agent is preparing to execute a standard tool.	None - System remains silent.
PostToolUse	* (All tools)	N/A	Agent successfully executed a standard tool.	None - System remains silent.

Resolving the Tab Focus Bottleneck via OSC Sequences

The primary technical challenge in implementing the Interrupt Protocol is solving the "Which window needs me?" problem.⁶ When a developer is running multiple Warp Terminal instances, simply triggering a standard system notification is insufficient. If the notification merely alerts the user that "an agent requires permission," the user is still forced to manually click through numerous terminal tabs to locate the specific blocked session.¹²

Standard terminal notification mechanisms rely on Operating System Command (OSC) escape sequences. The sequence OSC 9 (ESC] 9 ; <body> BEL) is historically utilized to send a simple body-text notification, while the more robust OSC 777 (ESC] 777 ; notify ; <title> ; <body> BEL)

supports both titles and body text.¹⁴ While Warp Terminal natively parses these sequences to trigger desktop notifications, clicking a standard OSC 777 notification typically only brings the application itself into focus, failing to switch the interface to the precise tab containing the blocked agent.¹³

To achieve true tab-specific focus recovery, VibeOS bypasses standard sequence parsing and directly leverages Warp's internal environment variable architecture. Warp exposes a unique `WARP_SESSION_ID` variable within every active terminal session.¹³ Concurrently, the terminal supports a proprietary deep-linking URI scheme formatted as `warp://session/<id>`.¹³

By utilizing a robust command-line interface tool such as `terminal-notifier` on macOS, the VibeOS bash script can construct a notification payload that embeds this deep link directly into the notification's execution action.¹³ When the developer clicks the native macOS banner, the operating system executes the deep link, which forces Warp to immediately bring the application to the foreground, locate the window containing that specific session, and automatically switch focus to the exact tab where the agent is stalled.¹³ This mechanism reduces the context-switching latency from several seconds of manual hunting to a single, frictionless click, maximizing developer freedom and preserving cognitive momentum.

Phase 2: The Performance Coach Retrospective

In high-performance software engineering, operational efficiency is not achieved through static configurations, but through continuous, recursive analysis of execution data. Standard AI coding sessions typically conclude abruptly, providing at best a passive summary of the files modified or a brief calculation of the session's duration.¹ For an architect prioritizing extreme token efficiency and workflow optimization, this represents a catastrophic loss of actionable telemetry.

Every token spent instructing an AI model incurs both a financial cost and, more importantly, a context-window cost.⁵ Wasted tokens lead to premature context exhaustion, which forces the developer to utilize the `/clear` command, purging the model's working memory and necessitating a disruptive re-initialization of the project state.¹

To enforce continuous workflow optimization, VibeOS completely replaces the standard termination sequence. It introduces the "Performance Coach," an interactive, data-driven retrospective protocol triggered automatically by the `SessionEnd` lifecycle hook.³

The Transcript Analysis Engine and Token Metrics

Throughout its execution lifecycle, Claude Code meticulously logs every interaction, tool execution, latency metric, and token calculation into a highly structured JSON Lines (.jsonl) transcript file.¹⁸ This file serves as an immutable cryptographic ledger of the entire development session. When the developer initiates the `/wrap` command or gracefully exits the

terminal, the SessionEnd hook fires, receiving the absolute file path of this ledger via the transcript_path property passed to standard input.³

The Performance Coach architecture dictates that a specialized subagent ingest this .jsonl file. Unlike standard conversational agents, this subagent is configured explicitly for log analysis, utilizing advanced parsing techniques to reconstruct the hierarchical span of the session.¹⁹ It extracts precise token consumption data, differentiating between input_tokens (the cost of the prompt), cache_read_input_tokens (the highly efficient retrieval of previously processed context), and completion_tokens (the actual code generated by the model).²²

By analyzing the ratio of these metrics, the engine identifies profound inefficiencies. For instance, if the transcript reveals a high volume of input_tokens relative to a low volume of cache_read_input_tokens, it indicates that the context window is frequently churning, likely due to the developer manually pasting large blocks of external documentation rather than utilizing persistent MCP server integrations.¹

The Vibe Score Metric

To quantify this operational efficiency, the Performance Coach calculates the "Vibe Score." This metric serves as a definitive grade for the session, evaluating the percentage of tokens successfully converted into pure architectural output versus the tokens wasted on re-explaining context, correcting hallucinations, or executing redundant tool calls.

The Vibe Score algorithm penalizes specific operational anti-patterns discovered within the transcript:

- **Prompt Churn:** Instances where the developer issues three or more sequential corrective prompts without the agent executing a successful tool call (e.g., "No, that's the wrong variable," followed immediately by "You missed the closing bracket"). This indicates a failure in initial specification.¹
- **Tool Loops:** Scenarios where the agent repeatedly executes a bash command, receives an error code, and attempts the exact same command without mutating the parameters, resulting in a severe token drain.¹⁹
- **Documentation Bloat:** Sessions where over 40% of the input tokens consist of raw API reference data pasted into the chat, rather than utilizing the /context command or a dedicated documentation reading tool.⁵

The following table illustrates the conceptual weighting of the Vibe Score deductions based on transcript analysis:

Inefficiency Anti-Pattern	Detection Mechanism via .jsonl Transcript	Vibe Score Deduction
---------------------------	---	----------------------

Sequential Prompt Corrections	Multiple UserPromptSubmit events without intervening PostToolUse successes.	-5 points per sequence
Unmodified Tool Execution Loops	Identical tool_input payloads in consecutive PreToolUse events followed by failures.	-10 points per loop
Low Cache Utilization	cache_read_input_tokens constitutes less than 20% of total input_tokens. ²²	-15 points
Context Warning Violations	Session continued executing tool calls after the 80% context threshold warning was triggered. ¹	-20 points (Critical)

The Coaching Dialogue and Dynamic Mutation

The true power of the Performance Coach lies not in passive scoring, but in active workflow mutation. Once the transcript analysis is complete, the SessionEnd hook clears the terminal screen and initiates an interactive dialogue with the developer.²¹

The sequence operates as follows:

1. **Metric Presentation:** The agent presents the calculated Vibe Score and provides a blunt, data-driven observation of the primary token waste. Example: *"Session terminated. Vibe Score: 78/100. Analysis indicates 42% of input tokens were consumed explaining the IndexedDB wrapper API."*
2. **Friction Inquiry:** The agent actively solicits human feedback: *"How did the execution feel? Identify any undocumented friction points or architectural disagreements."*
3. **System Proposal:** Based on the transcript parsing and the developer's feedback, the agent synthesizes a root cause and proposes an automated, permanent adjustment to the environment. Example: *"Proposal: The token drain was caused by repetitive explanations. Shall I append a persistent rule to CLAUDE.md mandating the use of the /context command for the IndexedDB module prior to implementation? (Y/N)"*
4. **Persistent Mutation:** If the developer approves the proposal, the hook script automatically mutates the project's CLAUDE.md file.

This recursive improvement model guarantees that the development environment functions as a learning system. Errors in communication, inefficient prompt structures, and missing documentation are systematically identified and permanently codified as rules, ensuring that

the same token-wasting friction is never repeated in subsequent sessions.¹

Phase 3: The Research-First Protocol

The most catastrophic waste of tokens, compute resources, and human capital does not occur during the writing of code, but during the implementation of an optimal syntax for a fundamentally suboptimal architecture.²⁵ High-performance software engineering mandates that strategic decisions precede tactical execution. If an AI agent is immediately unleashed to write code without a verified architectural foundation, the resulting technical debt necessitates massive refactoring cycles, completely negating the speed advantages of vibe coding.¹

To enforce architectural discipline, VibeOS introduces a strict "Phase 0" mandate for all new repositories. Before any code generation tools (such as Write or Edit) are permitted to execute, the system must trigger the "Stack Scout" research protocol.

The Stack Scout Subagent

The Stack Scout is an isolated, highly specialized subagent configured strictly for read-only operations.¹ It is granted access to powerful search utilities and external documentation via MCP servers (e.g., Puppeteer for browser automation, Context7 for technical documentation).¹ Its sole directive is to analyze the user's project requirements against the current state of technology, evaluate competitive tradeoffs, and generate a definitive architectural mandate.

Executing this research phase in a dedicated subagent is critical for context preservation. The process of searching the web, analyzing competitor repositories, and parsing extensive framework documentation consumes tens of thousands of tokens.¹ By isolating this process, the massive token cost remains entirely outside the primary development context window. The main implementation session remains pristine, populated only by the final, distilled architectural decisions, saving an estimated 8,000 to 15,000 tokens per project initialization.¹

The Technology Decision Record (TDR) Output

The execution of the Stack Scout agent culminates in the autonomous generation of a Technology Decision Record (TDR).²⁶ Traditional Architecture Decision Records (ADRs) are often written retrospectively by human engineers to document why a system was built a certain way.²⁶ In the VibeOS paradigm, the TDR serves as a proactive, predictive contract. It is an AI-generated artifact produced *before* implementation begins.²⁷

The developer must explicitly review and approve the TDR. Once approved, it is committed to the repository's documentation folder (e.g., docs/tdr-001-tech-stack.md) and persistently referenced by the primary CLAUDE.md configuration file.²⁶ This ensures that all subsequent implementation agents operate within a strictly defined technical boundary, drastically

reducing the probability of architectural hallucinations.

The TDR format strictly enforces the documentation of:

- **Context:** The technical requirements, performance targets, and business goals driving the project.
- **Options Considered:** A rigorous comparative analysis of the leading technological paradigms available.
- **Decision:** The definitive technology stack chosen for implementation.
- **Consequences:** The projected impact on token overhead, deployment complexity, hardware access, and overall application performance.

Case Study: High-Performance Travel Organizer in 2026

To fully demonstrate the analytical depth and necessity of the Stack Scout protocol, consider the initialization of a mobile-first "Travel Organizer" application in the year 2026. The developer tasks the agent to evaluate the current best-in-class tech stack, specifically requesting a comparison between a Progressive Web App (PWA) architecture utilizing IndexedDB versus a fully compiled React Native implementation.

The Stack Scout accesses current market data, noting that the proliferation of 5G connectivity and the maturation of WebAssembly have drastically altered the mobile performance landscape.²⁹ It analyzes the performance requirements of a travel application, which primarily involve offline data availability, GPS access, and rapid visual rendering, and maps these requirements against the capabilities of the two competing stacks.

The resulting comparative analysis highlights the following technical realities for 2026:

Evaluation Metric	PWA + IndexedDB Architecture (2026)	React Native Architecture (2026)	VibeOS Strategic Recommendation
Development Economics	50-70% lower overall development cost. Utilizes a single codebase built on standard DOM APIs. ²⁹	Higher cost and extended timelines. Requires maintenance of bridging layers and separate iOS/Android compilation targets. ²⁹	PWA. Strongly aligns with AI token-efficiency goals by eliminating the need to generate complex native bridging logic.

Performance (TTFB/LCP)	Sub-second initial load times enabled by advanced Service Worker caching and pervasive 5G networks. ³⁰	Exceptional runtime speed, but hampered by bloated initial bundle downloads and app store gatekeeping. ³¹	PWA. Superior for immediate user interaction, frictionless onboarding, and web-based discoverability.
Offline Data Persistence	IndexedDB provides highly robust, asynchronous, multi-gigabyte storage capabilities natively within the browser engine.	Async Storage or SQLite integrations require managing heavy native dependencies and build configurations.	PWA. Utilizing native browser APIs eliminates the need for the AI to manage external storage libraries, reducing dependency surface area.
Hardware Access	Expanded Browser APIs in 2026 securely expose 95% of standard hardware features, including camera and geolocation. ²⁹	Unrestricted, 100% direct access to all hardware via native compilation. ³¹	PWA. The functional requirements of a travel application (GPS, offline cache) do not necessitate the heavy GPU access that React Native provides.

The Stack Scout synthesizes this data and concludes that for a Travel Organizer, the slight edge React Native holds in raw, heavy-computation performance²⁹ is entirely irrelevant compared to the massive reduction in development overhead, bundle size, and maintenance burden offered by a modern PWA.²⁹

The TDR formally mandates the PWA + IndexedDB stack. Upon the developer's review and explicit approval of this document, the primary feature-developer agent is unlocked. It begins implementation according to the strict, pre-approved boundaries defined in the TDR, ensuring that no tokens are wasted exploring unsupported React Native libraries or debating architecture mid-session.

Architectural Deliverables and Implementation

Specifications

The physical implementation of VibeOS requires a strict directory structure and precise shell scripting to integrate seamlessly with the Claude Code plugin architecture.¹¹ The system is distributed as a local plugin, allowing the complex hook logic and subagent definitions to be packaged and version-controlled independently of the target project's business logic.¹¹

The following sections detail the core configuration files, bash execution scripts, and prompt templates that constitute the VibeOS environment.

1. Plugin Architecture and hooks.json Schema

The directory tree must be structured to expose the manifest, the event listeners, the executable scripts, and the agent prompts to the Claude Code orchestrator.¹¹ The root directory contains the .claude-plugin folder for metadata, while the functional components are organized into standard directories.¹¹

Directory Path	Component Type	Architectural Purpose
.claude-plugin/plugin.json	Manifest	Defines VibeOS metadata, versioning, and environment requirements. ¹¹
hooks/hooks.json	Configuration	The core event routing table binding Claude Code lifecycle events to local bash scripts. ³
hooks/scripts/notify-warp.sh	Executable Script	Bash logic parsing JSON standard input and executing OSC 777 native notifications. ³
hooks/scripts/coach-retro.sh	Executable Script	Bash logic extracting the transcript_path and launching the retrospective dialogue. ²⁰
agents/stack-scout.md	Agent Definition	The Phase 0 research agent prompt, governing

		the creation of the TDR. ¹¹
agents/performance-coach .md	Agent Definition	The transcript analysis agent prompt, responsible for calculating the Vibe Score. ¹¹

The hooks/hooks.json file is the central nervous system of VibeOS. It binds the lifecycle events to the shell scripts. It utilizes strict regex matchers to ensure the notify-warp.sh script only fires during the targeted attention economy events, preserving silence during standard execution.³

JSON

```
{
  "hooks": {
    "Notification":
  },
  "PostToolUseFailure":
  },
  "SessionEnd":
  }
}
```

2. The "Notify" Execution Script

The notify-warp.sh script is the execution engine of the Interrupt Protocol. When a monitored event fires, Claude Code passes a JSON payload representing the event context to the script via standard input (stdin).³

The script utilizes jq to parse this payload, extracting the notification_type to determine the context of the interruption.³³ Crucially, to solve the tab-focus problem discussed in Phase 1, the script captures the WARP_SESSION_ID environment variable and constructs a deep link.¹³ It then invokes the terminal-notifier utility, passing the deep link to the -execute flag. When the developer clicks the native macOS notification, the system executes the deep link, forcing

Warp to instantly focus the precise tab requiring intervention.¹³

Bash

```
#!/bin/bash
# VibeOS: Warp Terminal Native Notification Hook
# Reads JSON from stdin provided by the Claude Code Notification lifecycle event

# Ensure jq is installed for parsing the JSON payload
if ! command -v jq &> /dev/null; then
    echo "Error: jq is required for VibeOS notifications." >&2
    exit 0
fi

# Extract the JSON payload from standard input
INPUT=$(cat)
TYPE=$(echo "$INPUT" | jq -r '.notification_type // empty')
MESSAGE=$(echo "$INPUT" | jq -r '.message // "Attention required."')

# Determine if this was called via PostToolUseFailure (arg 1 = "error")
if [ "$1" == "error" ]; then
    TITLE="❌ Project: Error"
    BODY="A critical tool execution failed. Human intervention required."
else
    # Map notification types to specific alert configurations
    case "$TYPE" in
        "permission_prompt")
            TITLE="🔴 Project: Approval"
            BODY="Agent blocked. I need your Y/N approval to proceed."
            ;;
        "idle_prompt")
            TITLE="🟢 Project: Success"
            BODY="Complex task execution completed. Awaiting new instructions."
            ;;
        *)
            # Exit silently for any non-critical notifications to preserve Deep Work state
            exit 0
            ;;
    esac
fi
```

```
# Construct the deep link to focus the exact Warp tab utilizing the session ID
SESSION_ID="${WARP_SESSION_ID}"
DEEP_LINK="warp://session/$SESSION_ID"

# Dispatch the native macOS notification utilizing terminal-notifier
# The -execute flag ensures clicking the banner triggers the deep link
terminal-notifier \
  -title "$TITLE" \
  -message "$BODY" \
  -sound "Submarine" \
  -execute "open '$DEEP_LINK'"

# Exit with success code (0) to allow Claude Code to continue its execution loop
exit 0
```

3. The Performance Coach Template

The Performance Coach relies on a highly specialized system prompt passed to a Sonnet-based subagent. The bash script `coach-retro.sh` extracts the `transcript_path` from the `SessionEnd` standard input payload³ and invokes this agent, injecting the absolute path of the JSONL file for deep analysis.²¹

The following system prompt defines the `performance-coach.md` agent profile, mandating the direct, peer-to-peer, data-driven communication style required by elite performance experts.

name: performance-coach description: Analyzes the session transcript to calculate Vibe Score and propose permanent workflow optimizations. model: claude-3-5-sonnet-latest

Role

You are the VibeOS Performance Coach, an expert in LLM workflow optimization, token efficiency metrics, and high-performance software architecture.

Input Data

You will receive the raw .jsonl transcript file of the just-completed Claude Code session.

Task Execution

1. **Token Analysis:** Parse the message.usage fields within the transcript. Calculate the "Vibe Score" (0-100). The score must be based on the ratio of output tokens representing final code versus input tokens consumed by repeated context loading, prompt corrections, and tool execution failures.
2. **Friction Identification:** Scan the execution logs to identify any segment where the developer was forced to issue more than two corrective prompts to achieve a single goal, or where you executed the same failing shell command repeatedly.
3. **Interactive Dialogue:** Present the user with the following concise terminal output:
 - The final Vibe Score (e.g., "Vibe Score: 82/100").
 - A direct, blunt observation of the largest token waste identified in the session.
 - An actionable question: "How did this session feel? Identify any friction points or architectural disagreements."
4. **Rule Proposal:** Wait for the developer's response. Based on their input, synthesize a specific, concise rule designed to be added to the project's CLAUDE.md file to prevent this friction permanently.
 - Example format: "Proposal: Add the rule 'Always use the /context command for API docs before writing implementation logic.' Shall I append this to CLAUDE.md? (Y/N)"

Tone Constraints

Be highly direct, peer-to-peer, and strictly data-driven. Do not apologize for errors. Focus entirely on reducing mental overhead and maximizing architectural token yield. Treat the user as an expert Web Performance Google Developer Expert (GDE).

Day 1 Usage Guide and Operational Directives

To successfully initiate a new project under the VibeOS paradigm and guarantee maximum token efficiency, the developer must abandon legacy linear workflows and strictly adhere to the Research-First and Attention Economy protocols. The following operational directives outline the day one initialization sequence.

Step 1: Environment Initialization and Verification

Initialize the local environment by installing the VibeOS plugin via the Claude Code command-line interface. Verify that the Notification and SessionEnd hooks have successfully bound to the project space, ensuring the Interrupt Protocol is active.

Bash

```
claude plugin install /path/to/claude-plugin-vibe-os  
claude /hooks # Visually verify matchers for 'permission_prompt' and 'idle_prompt'
```

Step 2: Triggering Phase 0 (Research-First Mandate)

Do not initiate any code generation or file creation commands. Immediately invoke the Stack Scout agent to evaluate the conceptual project goals against current market data.

Bash

```
claude /agent stack-scout "Evaluate the optimal tech stack for a high-performance Travel Organizer app. Exhaustively compare PWA + IndexedDB versus React Native focusing on 2026 performance metrics."
```

Step 3: Background Multitasking and Deep Work

Once the Stack Scout is deployed, immediately switch focus to another window (e.g., a browser profiling tool or a design application) and begin secondary Deep Work. The terminal environment will remain completely silent. Rely entirely on the VibeOS Interrupt Protocol. The native macOS alert will only emit when the Stack Scout has successfully compiled the Technology Decision Record (TDR) and is awaiting final human authorization.

Step 4: TDR Review and Architectural Approval

Click the native notification banner to trigger the deep link, instantly returning focus to the precise Warp Terminal tab executing the session. Review the generated docs/tdr-001-tech-stack.md document. Analyze the provided data regarding development cost savings, bundle size implications, and offline capabilities of the recommended architectural approach.

Step 5: Proceed to Bounded Implementation

Upon the explicit approval of the TDR, initialize the primary feature developer workflow. The implementation agent will now operate strictly within the architectural boundaries codified in the TDR, effectively eliminating token-heavy exploratory hallucinations and ensuring high-performance execution.

Step 6: Retrospective Optimization and Termination

Upon concluding the primary work block, utilize the `/wrap` command or issue an EOF signal (Ctrl+D) to gracefully terminate the session and trigger the SessionEnd hook. Engage with the Performance Coach agent to review the calculated Vibe Score, identify operational inefficiencies, and authorize dynamic mutations to the CLAUDE.md rule set. This final step guarantees that any friction encountered during day one is permanently eradicated from all subsequent development sessions.

Works cited

1. vibe-coding-team-guide.docx
2. My Claude Code Setup - Pedro H. C. Sant'Anna, accessed February 14, 2026, <https://psantanna.com/claude-code-my-workflow/workflow-guide.html>
3. Hooks reference - Claude Code Docs, accessed February 14, 2026, <https://code.claude.com/docs/en/hooks>
4. Automate workflows with hooks - Claude Code Docs, accessed February 14, 2026, <https://code.claude.com/docs/en/hooks-guide>
5. Manage costs effectively - Claude Code Docs, accessed February 14, 2026, <https://code.claude.com/docs/en/costs>
6. Support for OSC 9 Desktop Notifications · Issue #7896 · warpdotdev/Warp - GitHub, accessed February 14, 2026, <https://github.com/warpdotdev/Warp/issues/7896>
7. Show HN: Warp, a Rust-based terminal - Hacker News, accessed February 14, 2026, <https://news.ycombinator.com/item?id=30921231>
8. How Warp Works, accessed February 14, 2026, <https://www.warp.dev/blog/how-warp-works>
9. Claude Code Notifications: Get Alerts When Tasks Finish (Hooks Setup) | alexop.dev, accessed February 14, 2026, <https://alexop.dev/posts/claude-code-notification-hooks/>
10. disler/claude-code-hooks-mastery - GitHub, accessed February 14, 2026, <https://github.com/disler/claude-code-hooks-mastery>
11. Plugins reference - Claude Code Docs, accessed February 14, 2026, <https://code.claude.com/docs/en/plugins-reference>
12. Option to show macOS banner notifications even when Warp is in the foreground · Issue #8439 - GitHub, accessed February 14, 2026, <https://github.com/warpdotdev/warp/issues/8439>
13. Session ID Environment Variable and Deep Link Support · Issue #8611 · warpdotdev/Warp, accessed February 14, 2026, <https://github.com/warpdotdev/warp/issues/8611>
14. Desktop Notifications - Warp, accessed February 14, 2026, <https://docs.warp.dev/terminal/more-features/notifications>
15. Official Warp terminal integration for Claude Code - native notifications and more - GitHub, accessed February 14, 2026, <https://github.com/warpdotdev/claude-code-warp>

16. Desktop notifications (OSC 9/99/777) · Issue #499 · gdamore/tcell - GitHub, accessed February 14, 2026, <https://github.com/gdamore/tcell/issues/499>
17. How to Get macOS Notifications for Long-Running Processes (Even over SSH!), accessed February 14, 2026, <https://dev.to/jfpio/how-to-get-macos-notifications-for-long-running-processes-even-over-ssh-154d>
18. Is there a way to have Claude Code search the current session's chat history? - Reddit, accessed February 14, 2026, https://www.reddit.com/r/ClaudeCode/comments/1pa0s0h/is_there_a_way_to_have_claude_code_search_the/
19. Finally got observability working for Claude Code: how the hooks actually work - Reddit, accessed February 14, 2026, https://www.reddit.com/r/Anthropic/comments/1qd1rto/finally_got_observability_working_for_claude_code/
20. claude-code-hooks - Skill | Smithery, accessed February 14, 2026, <https://smithery.ai/skills/filipexyz/claude-code-hooks>
21. Claude Code Hook Development Skill - Install to .claude/skills/hook-development/ · GitHub, accessed February 14, 2026, <https://gist.github.com/alexfazio/653c5164d726987569ee8229a19f451f>
22. How to Calculate Your Claude Code Context Usage - Codelynx, accessed February 14, 2026, <https://codelynx.dev/posts/calculate-claude-code-context>
23. Created a script to calculate token usage for a Copilot session for providing an accurate estimate of the real cost when running it via Claude Code : r/GithubCopilot - Reddit, accessed February 14, 2026, https://www.reddit.com/r/GithubCopilot/comments/1o764da/created_a_script_to_calculate_token_usage_for_a/
24. Claude Code now supports hooks : r/ClaudeAI - Reddit, accessed February 14, 2026, https://www.reddit.com/r/ClaudeAI/comments/1loodjn/claude_code_now_supports_hooks/
25. I've used AI to write 100% of my code for 1+ year as an engineer. 13 no-bs lessons - Reddit, accessed February 14, 2026, https://www.reddit.com/r/ClaudeCode/comments/1qxivbt/ive_used_ai_to_write_100_of_my_code_for_1_year_as/
26. AI generated Architecture Decision Records (ADR) - Dennis Adolphi, accessed February 14, 2026, <https://adolphi.dev/blog/ai-generated-adr/>
27. Agent Decision Records (AgDR) - A standard for documenting technical decisions made by AI coding agents. Extends ADR for the age of AI-assisted development. - GitHub, accessed February 14, 2026, <https://github.com/me2resh/agent-decision-record>
28. ADR process - AWS Prescriptive Guidance, accessed February 14, 2026, <https://docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/adr-process.html>
29. Progressive Web Apps vs Native Apps: Which Should You Choose in 2026?, accessed February 14, 2026,

- <https://vofoxsolutions.com/progressive-web-apps-vs-native-apps-in-2026>
30. Rise of PWAs vs. Native Apps | Which Should You Build in 2026? - Webshark Corporation, accessed February 14, 2026, <https://www.webshark.tech/blogs/the-rise-of-pwas-vs-native-apps-which-should-you-build-in-2026/>
 31. Hybrid vs Native vs PWA: App Development Guide 2026 - Elsner Technologies, accessed February 14, 2026, <https://www.elsner.com/hybrid-vs-native-vs-pwa-app-development/>
 32. Create plugins - Claude Code Docs, accessed February 14, 2026, <https://code.claude.com/docs/en/plugins>
 33. A lightweight macOS desktop notification hook for Claude Code that displays native notifications with customizable system sounds when events occur during AI coding sessions. - GitHub, accessed February 14, 2026, <https://github.com/wyattjoh/claude-code-notification>
 34. [Feature Request] Make transcript path more globally available to bash commands · Issue #5531 · anthropics/claude-code - GitHub, accessed February 14, 2026, <https://github.com/anthropics/claude-code/issues/5531>